1. Problem Title & Source

Title: **Minimum Additional Points to Form a Square**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Clear Version)

In a geometric puzzle, you are given **two points on a 2D plane**.

Each point represents a **corner of a potential square**.

Your task is to determine **how many additional points are required to form a valid square using these two points**.

Important understanding:

- A square has **4 vertices**.
- You are currently given **2 vertices**.
- Depending on their positions, you may need **0, 1, or 2 additional points** to complete the square.

Your goal is to determine the **minimum number of additional points required**.

3. Examples

Example 1

Input

0 0

0 1

Output

2

Explanation

The given points represent **two adjacent vertices of a square**.

Missing vertices:

(1,0)

(1,1)

Additional points required:

2

Example 2

Input

0 0

1 1

Output

2

Explanation

These points form a **diagonal of a square**.

Missing points:

(0,1)

(1,0)

Additional points required:

2

4. Constraints

-100 ≤ x,y ≤ 100

Observations

- Points may form either:
  - **side of square**
  - **diagonal of square**

5. Core Concept (Pattern)

Coordinate Geometry

Key idea:

Use **distance relationships between points**.

6. Thought Process

Step 1 — Identify Relationship

Two points can represent:

1. Adjacent corners
2. Diagonal corners

Step 2 — Adjacent Case

Example

(0,0) and (0,1)

We can extend perpendicular direction to form square.

Missing points = **2**

Step 3 — Diagonal Case

Example

(0,0) and (1,1)

Midpoint logic gives remaining points.

Missing points = **2**

Observation

Given only **two points**, we always require:

2 additional points

to form a square.

7. Visualization

Example

```
(0,0)   (1,0)
  +-------+
  |       |
  |       |
  +-------+
(0,1)   (1,1)
```

Given points:

(0,0)

(0,1)

Missing:

(1,0)

(1,1)

8. Pseudocode

```
read x1 y1
read x2 y2

print 2
```

9. Code Implementation

Python

```
x1, y1 = map(int,input().split())
x2, y2 = map(int,input().split())

print(2)
```

Java

```
import java.util.*;

class Main {
```

```
    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        int x1 = sc.nextInt();
        int y1 = sc.nextInt();

        int x2 = sc.nextInt();
        int y2 = sc.nextInt();

        System.out.println(2);
    }
}
```

10. Time & Space Complexity

Time Complexity

O(1)

Space Complexity

O(1)

11. Common Mistakes

1. Trying to calculate square sides unnecessarily
2. Overcomplicating geometry
3. Attempting distance calculations
4. Writing unnecessary loops

12. Detailed Dry Run

Input

(0,0)

(1,1)

Square needs:

4 vertices

Currently available:

2

Missing:

2

Output

2

13. Real-Life Applications

Geometry-based problems appear in:

- Computer graphics
- Game development
- Map coordinate systems
- Image processing

14. Common Traps

1. Overthinking geometry
2. Assuming complex calculations needed
3. Misinterpreting problem requirements
4. Trying to find exact coordinates

15. Related Problems

Geometry problems:

LC 593 Valid Square

LC 149 Max Points on a Line

LC 836 Rectangle Overlap

16. Alternate Approaches

Approach	Complexity	Notes
Direct observation	O(1)	optimal
Distance calculation	unnecessary	overkill

17. Why This Solution Works

A square always requires **4 vertices**.

Since only **2 points are provided**, the remaining number of points required will always be:

4 − 2 = 2

18. Variations / Follow-Up Questions

Possible interview variations:

1. Find **coordinates of missing points**
2. Check if **4 points form a square**
3. Minimum points to form **rectangle**
4. Square detection from **multiple points**